



## **ATIVIDADE ACADÊMICA**

### **SISTEMA DE TELEMETRIA PARA DECOLAGEM**

Relatório operacional

**ERICK SANTOS ALMEIDA**

São Paulo – SP

2026

**FIAP**

**ATIVIDADE ACADÊMICA**

**SISTEMA DE TELEMETRIA PARA DECOLAGEM**

Relatório operacional

Relatório apresentado à FIAP como atividade acadêmica, com foco na organização de dados de telemetria, desenvolvimento de algoritmo de verificação, análise energética, programação em Python, triagem assistida por inteligência artificial e reflexão ética e sustentável aplicada à exploração espacial.

**GITHUB:**

<https://github.com/nickinvalido>

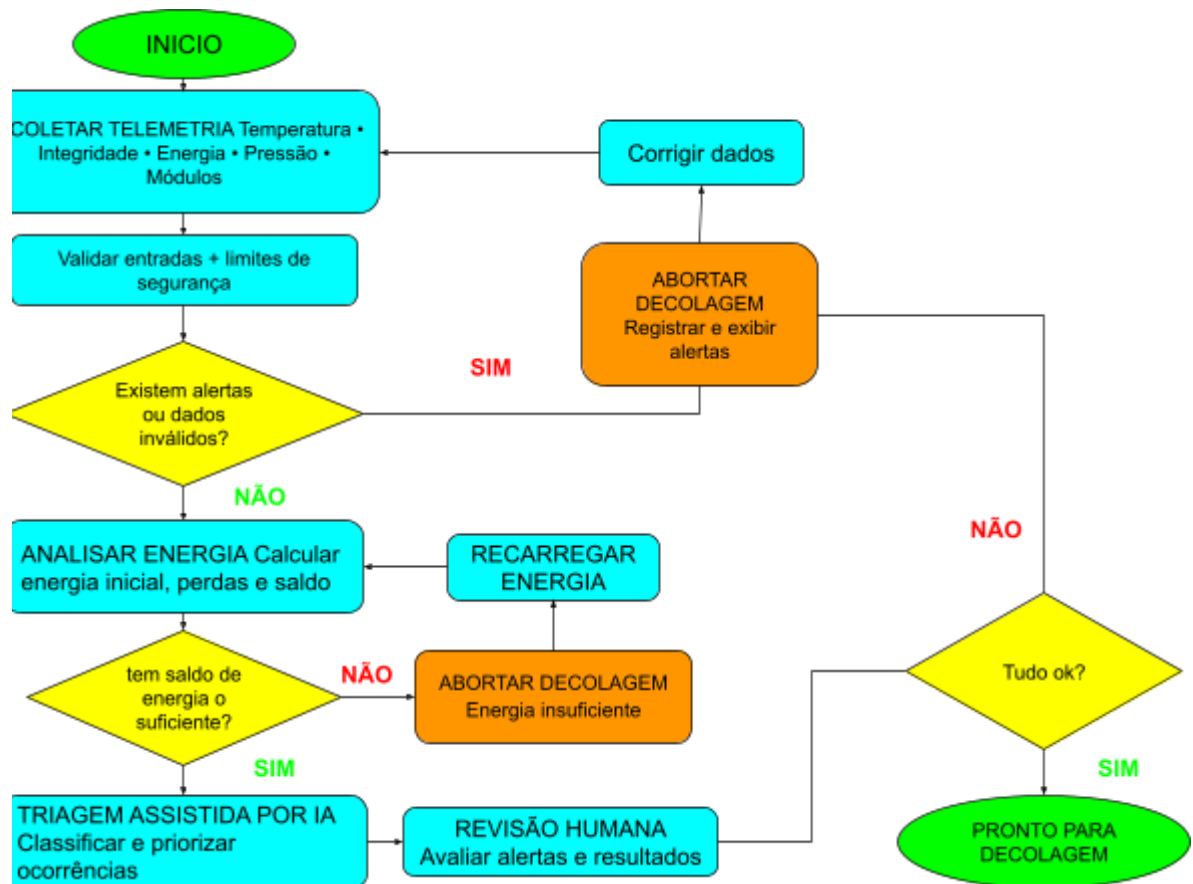
São Paulo - SP

2026

## FLUXOGRAMA

### SISTEMA DE TELEMETRIA PARA DECOLAGEM

Atividade acadêmica — FIAP



## **SUMÁRIO**

1. Objetivo
2. Organização e descrição da telemetria
3. Algoritmo de verificação
4. Análise energética
5. Testes
6. Script Python
7. Triagem assistida por IA
8. Reflexão ética e sustentável
10. Referência do projeto

### **1. OBJETIVO**

Desenvolver um script Python funcional que interprete dados de telemetria, verifique condições de segurança e decida entre PRONTO PARA DECOLAGEM e DECOLAGEM ABORTADA.

## **2. ORGANIZAÇÃO E DESCRIÇÃO DA TELEMETRIA**

| Dado                        | Critério   |
|-----------------------------|------------|
| Temperatura interna         | 10 a 35 °C |
| Temperatura externa         | 20 a 50 °C |
| Integridade estrutural      | 0 ou 1     |
| Nível de energia            | mínimo 40% |
| Pressão dos tanques         | 1 a 4 bar  |
| Status dos módulos críticos | 0 ou 1     |

## **3. ALGORITMO DE VERIFICAÇÃO**

O programa lê os parâmetros de telemetria, verifica cada faixa de segurança, calcula a condição energética e acumula alertas. Se não houver alertas, o sistema retorna PRONTO PARA DECOLAGEM; caso contrário, retorna DECOLAGEM ABORTADA.

Pseudocódigo

PSEUDOCÓDIGO — SISTEMA DE TELEMETRIA

INÍCIO

Exibir critérios de segurança

Ler temperatura interna

Ler temperatura externa

Ler integridade

Ler nível de energia

Ler pressão

Ler status dos módulos críticos

Ler capacidade total de energia (kWh)

Ler carga atual (%)

Ler consumo estimado na decolagem (kWh)

Ler perdas energéticas (%)

Criar lista de alertas vazia

Verificar temperatura interna:

SE menor que 10 OU maior que 35

adicionar alerta

FIMSE

Verificar temperatura externa:

SE menor que 20 OU maior que 50

adicionar alerta

FIMSE

SE integridade = 0

adicionar alerta

FIMSE

SE nível de energia < 40%

adicionar alerta

FIMSE

SE pressão < 1 OU pressão > 4

adicionar alerta

FIMSE

SE módulos críticos = 0

adicionar alerta

FIMSE

$\text{energia\_inicial} \leftarrow \text{capacidade\_total} \times (\text{carga\_atual} / 100)$

$\text{energia\_perdida} \leftarrow \text{energia\_inicial} \times (\text{perdas\_energeticas} / 100)$

$\text{energia\_apos\_perdas} \leftarrow \text{energia\_inicial} - \text{energia\_perdida}$

$\text{saldo\_apos\_decolagem} \leftarrow \text{energia\_apos\_perdas} -$

$\text{consumo\_decolagem}$  SE  $\text{saldo\_apos\_decolagem} < 0$

adicionar alerta de energia insuficiente

FIMSE

SE não houver alertas

exibir "PRONTO PARA DECOLAGEM"

SENÃO

exibir "DECOLAGEM ABORTADA"

exibir todos os alertas

FIMSE  
FIM

#### 4. ANÁLISE ENERGÉTICA

A atividade solicita calcular a autonomia/condição energética considerando capacidade total, carga atual, consumo estimado na decolagem e perdas energéticas.

Fórmulas:

Energia inicial = capacidade total  $\times$  (carga atual / 100)

Energia perdida = energia inicial  $\times$  (perdas / 100)

Energia após perdas = energia inicial - energia perdida

Saldo após decolagem = energia após perdas - consumo da decolagem  
No teste aprovado: 100 kWh de capacidade, 80% de carga, 10% de perdas e 20 kWh de consumo. Energia inicial =

80 kWh; perda = 8 kWh; energia após perdas = 72 kWh; saldo = 52 kWh. Portanto, a condição energética é suficiente.

No teste rejeitado: 100 kWh de capacidade, 30% de carga, 10% de perdas e 35 kWh de consumo. Energia inicial =

30 kWh; perda = 3 kWh; energia após perdas = 27 kWh; saldo = -8 kWh. Portanto, a energia é insuficiente.

#### 5. TESTES

Teste aprovado: 25 °C; 30 °C; integridade 1; energia 80%; pressão 2,5 bar; módulos 1; capacidade 100 kWh; carga 80%; consumo 20 kWh; perdas 10%.

Resultado esperado: PRONTO PARA DECOLAGEM.

Teste rejeitado: 5 °C; 55 °C; integridade 0; energia 30%; pressão 5 bar; módulos 0; capacidade 100 kWh; carga

30%; consumo 35 kWh; perdas 10%.

Resultado esperado: DECOLAGEM ABORTADA, com múltiplos alertas.



## 6. SCRIPT PYTHON

```
# SISTEMA DE TELEMETRIA PARA DECOLAGEM
print("===== CRITERIOS PARA DECOLAGEM =====")
print("Temperatura interna: 10 a 35 °C")
print("Temperatura externa: 20 a 50 °C")
print("Integridade: 0 ou 1")
print("Nível de energia: mínimo 40%")
print("Pressão: 1 a 4 bar")
print("Módulos críticos: 0 ou 1")
print("=====")
print("===== DIGITE OS VALORES PEDIDOS =====")
print("=====")
def obter_telemetria():
    return {
        'temperatura': temperatura,
        'temperatura_externa': temperatura_externa,
        'integridade': integridade,
        'nivel_energia': energia,
        'pressao': pressao,
        'status_modulos_criticos': status_modulos_criticos,
        'capacidade_total': capacidade_total,
        'carga_atual': carga_atual,
        'consumo_decolagem': consumo_decolagem,
        'perdas_energeticas': perdas_energeticas,
    }
# TEMPERATURA INTERNA
while True:
    try:
        temperatura = float(input("Digite a temperatura interna da nave: ")) if temperatura >= 0:
            break
        print("A temperatura não pode ser negativa.")
    except ValueError:
        print("Insira um número válido!!!")
# TEMPERATURA EXTERNA
while True:
    try:
        temperatura_externa = float(input("Digite a temperatura externa da nave: ")) if temperatura_externa >= 0:
            break
        print("A temperatura não pode ser negativa.")
    except ValueError:
        print("Insira um número válido!!!")
# INTEGRIDADE
while True:
    try:
        integridade = int(input("Digite a integridade estrutural (0 ou 1): ")) if integridade in [0, 1]:
            break
        print("Integridade deve ser 0 ou 1.")
    except ValueError:
        print("Insira um número válido!!!")
# NÍVEL DE ENERGIA
while True:
    try:
        energia = int(input("Digite o nível de energia (%): "))
        if 0 <= energia <= 100:
            break
        print("Energia deve estar entre 0 e 100%.")
    except ValueError:
        print("Insira um número válido!!!")
# PRESSÃO
while True:
    try:
        pressao = float(input("Digite a pressão dos tanques (bar): ")) if pressao >= 0:
            break
        print("A pressão precisa ser positiva.")
    except ValueError:
```

```

print("Insira um número válido!!!")
# STATUS DOS MÓDULOS CRÍTICOS
while True:
    try:
        status_modulos_criticos = int(
            input("Digite o status dos módulos críticos (1 para OK, 0 para falha): ") )
        if status_modulos_criticos in [0, 1]:
            break
        print("Status dos módulos críticos deve ser 0 ou 1.")
    except ValueError:
        print("Insira um número válido!!!")
# DADOS PARA ANÁLISE ENERGÉTICA
while True:
    try:
        capacidade_total = float(input("Digite a capacidade total de energia (kWh): ")) if capacidade_total > 0:
            break
        print("A capacidade total deve ser maior que zero.")
    except ValueError:
        print("Insira um número válido!!!")
    while True:
        try:
            carga_atual = float(input("Digite a carga atual da bateria (%): ")) if 0 <= carga_atual <=
            100:
                break
            print("A carga atual deve estar entre 0 e 100%.")
        except ValueError:
            print("Insira um número válido!!!")
    while True:
        try:
            consumo_decolagem = float(
                input("Digite o consumo estimado na decolagem (kWh): ")
            )
            if consumo_decolagem >= 0:
                break
            print("O consumo não pode ser negativo.")
        except ValueError:
            print("Insira um número válido!!!")
    while True:
        try:
            perdas_energeticas = float(
                input("Digite as perdas energéticas (%): ")
            )
            if 0 <= perdas_energeticas <= 100:
                break
            print("As perdas devem estar entre 0 e 100%.")
        except ValueError:
            print("Insira um número válido!!!")
    def analisar_energia(telemetria):
        capacidade = telemetria['capacidade_total']
        carga = telemetria['carga_atual']
        consumo = telemetria['consumo_decolagem']
        perdas = telemetria['perdas_energeticas']
        energia_inicial = capacidade * (carga / 100)
        energia_perdida = energia_inicial * (perdas / 100)
        energia_apos_perdas = energia_inicial - energia_perdida
        saldo_apos_decolagem = energia_apos_perdas - consumo
        print("\n===== ANÁLISE ENERGÉTICA =====")
        print(f"Capacidade total: {capacidade:.2f} kWh")
        print(f"Carga atual: {carga:.2f}%")
        print(f"Energia disponível inicialmente: {energia_inicial:.2f} kWh") print(f"Perdas energéticas:
        {perdas:.2f}%")
        print(f"Energia perdida: {energia_perdida:.2f} kWh")
        print(f"Energia após perdas: {energia_apos_perdas:.2f} kWh")
        print(f"Consumo estimado na decolagem: {consumo:.2f} kWh")
        print(f"Saldo energético após a decolagem: {saldo_apos_decolagem:.2f} kWh") if
        saldo_apos_decolagem >= 0:
            print("ANÁLISE: Energia suficiente para a decolagem.")
    return True

```

```

else:
    print("ANÁLISE: Energia insuficiente para a decolagem.")
    return False
def verificar_decolagem(telemetria):
    alertas = []
    # Verificações de telemetria
    if telemetria['temperatura'] > 35:
        alertas.append("Alerta: Temperatura interna acima do limite seguro!")
    if telemetria['temperatura'] < 10:
        alertas.append("Alerta: Temperatura interna abaixo do limite seguro!")
    if telemetria['temperatura_externa'] < 20:
        alertas.append("Alerta: Temperatura externa abaixo do limite seguro!")
    if telemetria['temperatura_externa'] > 50:
        alertas.append("Alerta: Temperatura externa acima do limite seguro!")
    if telemetria['integridade'] == 0:
        alertas.append("Alerta: Integridade estrutural corrompida!")
    if telemetria['nivel_energia'] < 40:
        alertas.append("Alerta: Nível de energia abaixo do mínimo de 40%!")
    if telemetria['pressao'] < 1:
        alertas.append("Alerta: Pressão abaixo do limite seguro!")
    if telemetria['pressao'] > 4:
        alertas.append("Alerta: Pressão acima do limite seguro!")
    if telemetria['status_modulos_criticos'] == 0:
        alertas.append("Alerta: Falha em módulos críticos!")
    # Análise energética
    energia_suficiente = analisar_energia(telemetria)
    if not energia_suficiente:
        alertas.append("Alerta: Energia insuficiente para a decolagem!") # Decisão final
    if len(alertas) == 0:

return "PRONTO PARA DECOLAGEM"
resultado = "DECOLAGEM ABORTADA!\n"
resultado += "ALERTAS ENCONTRADOS:\n"
for alerta in alertas:
    resultado += f"- {alerta}\n"
return resultado
telemetria = obter_telemetria()
resultado = verificar_decolagem(telemetria)
print("\n===== RESULTADO DA TELEMETRIA =====")
print("=====")
print(f"> temperatura interna: {telemetria['temperatura']} °C")
print(f"> temperatura externa: {telemetria['temperatura_externa']} °C")
print(f"> integridade: {telemetria['integridade']}")
print(f"> nível de energia: {telemetria['nivel_energia']} %")
print(f"> pressão: {telemetria['pressao']} bar")
print(f"> módulos críticos: {telemetria['status_modulos_criticos']}")
print(f"> capacidade total: {telemetria['capacidade_total']} kWh")
print(f"> carga atual: {telemetria['carga_atual']} %")
print(f"> consumo na decolagem: {telemetria['consumo_decolagem']} kWh")
print(f"> perdas energéticas: {telemetria['perdas_energeticas']} %")
print("=====")
print(f"RESULTADO FINAL: {resultado}")

```

## **7. TRIAGEM ASSISTIDA POR IA**

A inteligência artificial pode atuar como uma camada de apoio à análise da telemetria, sem substituir os critérios determinísticos de segurança já definidos no projeto. A partir dos dados coletados, um sistema de IA pode auxiliar na organização, classificação e priorização das ocorrências, identificando padrões anormais e agrupando os registros por nível de atenção.

No contexto deste projeto, a triagem pode receber como entrada temperatura interna e externa, integridade estrutural, nível de energia, pressão dos tanques, status dos módulos críticos e os parâmetros da análise energética. A IA poderia classificar os dados em normal, atenção e crítico. Essa classificação serviria para destacar rapidamente os parâmetros que merecem investigação humana.

Fluxo proposto: coleta da telemetria → validação dos valores → análise pelas regras de segurança → triagem assistida por IA → classificação e priorização dos alertas → revisão por responsável técnico → decisão final.

A IA também pode contribuir para detectar combinações de sinais que, isoladamente, parecem aceitáveis, mas que juntas podem indicar uma situação de risco. Em um sistema de segurança de decolagem, a decisão final deve permanecer vinculada às regras de segurança verificáveis e à supervisão humana.

## **8. REFLEXÃO ÉTICA E SUSTENTÁVEL**

A utilização de inteligência artificial em um sistema de telemetria espacial exige responsabilidade porque erros de classificação podem afetar pessoas, equipamentos e recursos de alto valor. Por isso, é necessário manter rastreabilidade das decisões, registrar os dados utilizados na triagem, estabelecer limites para a atuação automática e garantir que situações críticas possam ser revisadas por profissionais responsáveis.

Também é importante considerar possíveis vieses e falhas do modelo. Uma IA treinada com dados insuficientes ou pouco representativos pode deixar de reconhecer determinados padrões de risco. Assim, a solução deve ser testada com cenários aprovados e rejeitados, além de casos anômalos, e seus resultados

precisam ser comparados com os critérios determinísticos do sistema.

Do ponto de vista sustentável, a análise de telemetria pode contribuir para o uso mais eficiente de energia e recursos. O próprio cálculo de energia inicial, perdas energéticas, energia após perdas e saldo após a decolagem permite avaliar se uma operação possui margem energética adequada. Uma triagem inteligente pode ajudar a priorizar perdas, anomalias e condições que indiquem desperdício de recursos, contribuindo para decisões operacionais mais eficientes.

Na exploração espacial, sustentabilidade também envolve reduzir desperdícios, aumentar a confiabilidade dos equipamentos, prolongar a vida útil de sistemas e considerar os impactos ambientais associados às operações. A tomada de decisão deve buscar equilíbrio entre segurança, eficiência energética, confiabilidade tecnológica e responsabilidade ambiental.

## **9. REFERÊNCIA DO PROJETO**

Repositório GitHub do projeto. Disponível em:

<https://github.com/nickinvalido/projeto-telemetria-decolagem>

Documento elaborado para fins de atividade acadêmica da FIAP.